

```
In [ ]: !pip install -U langchain langchain-community langchain-google-genai google-generativeai faiss-cpu
```

```
In [6]: !pip install -q sentence-transformers
```

```
In [2]: import os
os.environ["NGROK_AUTH_TOKEN"] = ""
```

```
In [3]: import os
os.environ["GEMINI_API_KEY"] = ""
```

```
In [ ]: %%writefile app.py
import os
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_classic.chains import ConversationalRetrievalChain
from langchain_classic.memory import ConversationBufferMemory
from langchain_classic.prompts import PromptTemplate

from langchain_community.document_loaders import DataFrameLoader
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings

st.set_page_config(page_title="InsightForge BI Assistant", layout="wide")

# Using your verified API key
MY_GEMINI_KEY = os.getenv("GEMINI_API_KEY")

def process_data(file):
    df = pd.read_csv(file, encoding="latin1")
    stats = df.describe(include="all")
    return df, stats

@st.cache_resource
def setup_rag_system(df):
    loader = DataFrameLoader(df, page_content_column=df.columns[0])
    docs = loader.load()

    embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
    vectorstore = FAISS.from_documents(docs, embeddings)

    memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

    template = """You are InsightForge, an expert BI assistant. Use the context to answer analytical questions.
    Context: {context}
    Question: {question}
    Answer: """

    prompt = PromptTemplate(template=template, input_variables=["context", "question"])

    # CHANGED TO gemini-1.5-flash-Latest for maximum compatibility
    llm = ChatGoogleGenerativeAI(
        model="gemini-3-flash-preview",
        google_api_key=MY_GEMINI_KEY,
        temperature=0
    )

    return ConversationalRetrievalChain.from_llm(
        llm=llm,
        retriever=vectorstore.as_retriever(),
        memory=memory,
        combine_docs_chain_kwargs={"prompt": prompt}
    )

st.title("🚀 InsightForge: AI-Powered BI Assistant")

uploaded_file = st.file_uploader("Upload Business Dataset (CSV)", type="csv")

if uploaded_file:
    df, stats = process_data(uploaded_file)
    st.subheader("📊 Dataset Summary")
    st.dataframe(stats)

    col1, col2 = st.columns(2)
    with col1:
        if "Date" in df.columns and "Sales" in df.columns:
            st.write("**Sales Trends**")
            st.line_chart(df.groupby("Date")["Sales"].sum())
        if "Region" in df.columns and "Sales" in df.columns:
            st.write("**Regional Sales**")
            fig, ax = plt.subplots()
```

```
df.groupby("Region")["Sales"].sum().plot(kind="bar", ax=ax, color='skyblue')
st.pyplot(fig)

with col2:
    if "Product" in df.columns and "Sales" in df.columns:
        st.write("**Product Performance**")
        fig2, ax2 = plt.subplots()
        sns.boxplot(x="Product", y="Sales", data=df, ax=ax2)
        st.pyplot(fig2)
    if "Demographics" in df.columns:
        st.write("**Demographics Split**")
        st.bar_chart(df["Demographics"].value_counts())

st.divider()
st.subheader("💬 Ask the Data")

if "chain" not in st.session_state:
    st.session_state.chain = setup_rag_system(df)

query = st.text_input("Ask about trends or recommendations")

if query:
    with st.spinner("InsightForge is analyzing..."):
        result = st.session_state.chain.invoke({"question": query})
        st.write(f"**Answer:** {result['answer']}")
else:
    st.info("Upload a CSV dataset to start analysis.")
```

```
In [ ]: from pyngrok import ngrok
import threading
import os
import time

# Optional but recommended: kill old tunnels
ngrok.kill()

# Set auth token (better to store in env variable)
ngrok.set_auth_token(os.getenv("NGROK_AUTH_TOKEN"))

# Start Streamlit server
def run_streamlit():
    os.system("streamlit run app.py --server.port 8501 --server.address 0.0.0.0")

thread = threading.Thread(target=run_streamlit)
thread.start()

# Wait a bit so Streamlit actually starts
time.sleep(5)

# Create public tunnel
public_url = ngrok.connect(8501)

print("🚀 Streamlit App URL:", public_url)
```